PRACTICAL ASSIGNMENT

Software Re-engineering (SREE) | BS Software Engineering - Semester 7

Total Marks 50	**Group Size** 2 students
Mode Hands-on coding + viva	**Submission** Source code, tests and report

Assignment: Re-engineer a Legacy Order Processing System

You are part of a maintenance team taking over an existing Python order calculator. The code works for a basic example but contains duplicated logic, weak modularity and limited validation. Improve its maintainability **without changing the intended results for valid existing inputs**. Add one controlled corrective change and demonstrate its impact.

Starter project and tools

• Use the supplied **legacy_order_system.py** starter file (attached separately).

• Tools: Python 3, VS Code / PyCharm (or another Python editor), pytest or unittest, Git; diagrams.net or any UML tool.

• Before modifying code, run the original program and capture its output. Save a baseline Git commit.

Task 1 | Program comprehension and maintenance analysis - 10 marks

• Explain the flow of **order_total()** and **print_receipt()** using a flowchart or simple function-level diagram.

• Identify **at least four** maintainability issues / code smells, citing exact lines and why they matter.

• Classify these requests as corrective, adaptive, perfective or preventive: (a) reject negative quantities; (b) support a new delivery provider; (c) improve receipt readability; (d) add tests to guard against future regressions. Briefly justify each.

Task 2 | Safe refactoring - 15 marks

• Refactor repetitive category calculations; extract small functions for subtotal, discount and delivery fee; replace magic numbers with named constants.

• Improve naming, input clarity and documentation. Keep the same public functions, existing discount rates and delivery charges.

• Show before/after code excerpts and explain **three** refactoring decisions. Do not add new features under the label of refactoring.

Task 3 | Corrective maintenance and impact analysis - 10 marks

• Apply one explicit corrective change: a negative **price** or **quantity** must raise a **ValueError** rather than produce a misleading total.

• Document which functions, inputs, tests and outputs are affected. Draw a small dependency/call graph and explain the ripple effect.

ASSIGNMENT TASKS & ASSESSMENT

Continue using your original baseline to prove that refactoring preserved existing valid-input behavior.

Task 4 | Regression testing and demonstration - 10 marks

• Write at least **six automated tests**: regular customer, student discount, staff discount, standard delivery, express delivery and invalid negative input.

• For the first five cases, compare before/after totals for the same input. For invalid input, verify the new expected **ValueError**.

• Run the full test suite and include a screenshot or terminal output showing the results.

Task 5 | Reflection and SPE classification - 5 marks

• Explain in 120-180 words whether this system is closest to an **S-type, P-type or E-type** program and justify with assumptions.

• Identify one practical example of **Lehman's continuing change** and one of **increasing complexity** affecting an evolving order system.

Minimum test data

Scenario	Input / expected baseline output
Student + standard	Notebook: 2 x 300; USB: 1 x 800 | **1360.00**
Regular + no delivery	Notebook: 2 x 300 | **600.00**
Staff + express	USB: 1 x 800 | **1010.00**
Invalid price / quantity	Any negative price or quantity | **ValueError after the fix**

Submission checklist

• **Git repository / ZIP:** baseline code, refactored code, tests, and at least two meaningful commits.

• **Short PDF report (2-4 pages):** code smells, diagram, maintenance classifications, impact analysis, test evidence, SPE and Lehman reflection.

• **Live demonstration:** run the baseline, show updated code, execute tests and answer questions about changed behavior.

Marking rubric

Program comprehension and analysis	**10**
Safe refactoring	**15**
Corrective maintenance and impact analysis	**10**
Regression testing and demo	**10**
SPE and Lehman reflection	**5**
TOTAL	**50**

Academic integrity: Each pair must explain its own code and decisions. Submit original work; disclose any AI assistance used.